

tests/run-tests.sh — operator guide

Revision: 1 **Last modified:** 2026-07-01T00:00:00Z **Authority:** Helix Constitution §11.4.3 (topology-aware SKIP), §11.4.35 (inheritance gate), §11.4.135 (standing regression suite), §11.4.115 (RED_MODE polarity), §11.4.69 (sink-side security guards)

Companion (§11.4.18) to the in-source header/comment blocks in the script.

Overview / Purpose

The project's **structural + regression + security test suite** — the primary bash `tests/run-tests.sh` entry point. It does NOT drive the live data plane; it validates the repository layout, the constitution-inheritance gate, the standing §11.4.135 regression guards, the live-proxy security guards, and the environment/config/runtime/port/cache/VPN topology. Ports are classified **topology-aware** (§11.4.3): a port held by the project's own running container is the healthy PASS state; a port held by a non-project process is SKIP (not attributable to the proxy), not a false FAIL.

It is a self-contained bash script that shells out to sibling gate/guard scripts. A SKIP counts as neither pass nor fail; only a real FAIL makes the suite exit non-zero.

Usage

```
bash tests/run-tests.sh
```

No flags. Behaviour is influenced only by the environment (below) and by which sibling guard scripts are present + parseable.

Inputs

- **.env** (optional, gitignored) — sourced if present for HTTP_PROXY_PORT, SOCKS_PROXY_PORT, PROXY_ADMIN_PORT, CACHE_DIR, USE_VPN, VPN_OVPN_PATH, VPN_USERNAME, VPN_PASSWORD. Absent =

fresh-checkout topology → the `.env` checks SKIP and `.env.example` is validated instead.

- **RUN_STARTUP_TESTS** (default false) — when true, runs `./init --check` and `./start --dry-run`.
- **SKIP_LE_ISSUANCE_GUARD** (default 0) — when 1, skips the two expensive hermetic Let's Encrypt Phase-3 / Phase-5 issuance guards.
- Port defaults when `.env` is absent: HTTP 34128, SOCKS 34080, admin 34088.

Outputs

- Coloured per-test PASS/SKIP/FAIL lines and a final summary (Tests Run / Passed / Skipped / Failed).
- Exit 0 — no FAIL (skips allowed); exit 1 — one or more real FAIL.
- Regression/security guards it invokes write their own evidence files under `qa-results/` (this script itself writes none).

Side-effects

None of its own. It delegates to read-only gates/guards; the LE issuance guards it may invoke boot a hermetic stack (gated by `SKIP_LE_ISSUANCE_GUARD`). It runs `cd "$PROJECT_ROOT"` and, in `RUN_STARTUP_TESTS=true` mode, invokes `./init --check` and `./start --dry-run`.

Dependencies

- bash (set -euo pipefail), grep, awk, ss; podman for the topology-aware port + runtime checks; docker/podman-compose optionally for the compose-syntax check.
- Sibling scripts it runs: `tests/constitution_inheritance_gate.sh`; `tests/regression/*` guards (`log_dir_writable_test.sh`, `test_result_returns_zero_test.sh`, `port_topology_aware_test.sh`, `cache_cli_present_test.sh`, `comprehensive_admin_topology_test.sh`, `no_suspend_export_sibling_test.sh`, `external_egress_verdict_test.sh`, `proxy_conn_verdict_test.sh`, `ddos_flood_evidence_test.sh`, `benchmark_baseline_ratchet_test.sh`, `assert_egress_ip_host_unknown_test.sh`, `cert_analyzer_selfvalidation_test.sh`); `tests/letsencrypt/phase3_issuance_guard.sh`
 - `phase5_rotation_guard.sh`; `tests/security/proxy_acl_security.sh`.

Internal behaviour notes

- `test_result` uses assignment-form counters (`VAR=$((VAR+1))`), `NOT ((VAR++))`, and always return 0 — a post-increment from 0 returns exit 1 under `set -e` and would abort the whole suite (BUGFIX-0001 / BUGFIX-0003). Guarded by `tests/regression/test_result_returns_zero_test.sh`.
- Each regression guard is run twice: once as the GREEN guard and once with `RED_MODE=1` to prove the guard genuinely reproduces its defect (§11.4.115 / a RED that cannot reproduce is a §11.4.7 finding).
- The security guard uses `|| rc=$?` capture so a non-zero guard exit does not abort the `set -euo pipefail` suite before the code is read.

Related scripts

- `tests/pre_build_verification.sh` — the pre-build gate runner (runs the same inheritance gate).
- `tests/comprehensive-test.sh` — the live data-plane functional suite.
- `docs/scripts/log_dir_writable_test.md`, `docs/scripts/cache_cli_present_test.md` — companion docs for guards this suite registers.

Last verified

2026-07-01 — read from source; `sh -n/bash -n` parse discipline mirrored from the sibling guards. Behaviour documented from the script body, not executed here (the conductor runs the live suite).